

是个布尔表达式, 所以无论涉及什么实际类型, 无论 `w.size() > 10 && w != someNastyWidget` 导致什么, 它都必须与 `bool` 兼容。这是 `template doProcessing` 加诸于其类型参数 (type parameter) `T` 的隐式接口的一部分。`doProcessing` 要求的其他隐式接口: `copy` 构造函数、`normalize` 和 `swap` 也都必须对 `T` 型对象有效。

加诸于 `template` 参数身上的隐式接口, 就像加诸于 `class` 对象身上的显式接口一样真实, 而且两者都在编译期完成检查。就像你无法以一种“与 `class` 提供之显式接口矛盾”的方式来使用对象 (代码将通不过编译), 你也无法在 `template` 中使用“不支持 `template` 所要求之隐式接口”的对象 (代码一样通不过编译)。

请记住

- `classes` 和 `templates` 都支持接口 (interfaces) 和多态 (polymorphism)。
- 对 `classes` 而言接口是显式的 (explicit), 以函数签名为中心。多态则是通过 `virtual` 函数发生于运行期。
- 对 `template` 参数而言, 接口是隐式的 (implicit), 莫基于有效表达式。多态则是通过 `template` 具现化和函数重载解析 (function overloading resolution) 发生于编译期。

条款 42: 了解 typename 的双重意义

Understand the two meanings of `typename`.

提一个问题: 以下 `template` 声明式中, `class` 和 `typename` 有什么不同?

```
template<class T> class Widget;           //使用 "class"  
template<typename T> class Widget;       //使用 "typename"
```

答案: 没有不同。当我们声明 `template` 类型参数, `class` 和 `typename` 的意义完全相同。某些程序员始终比较喜欢 `class`, 因为可以少打几个字。其他人 (包括我) 比较喜欢 `typename`, 因为它暗示参数并非一定得是个 `class` 类型。少数开发人员在接受任何类型时使用 `typename`, 而在只接受用户自定义类型时保留旧式的 `class`。然而从 C++ 的角度来看, 声明 `template` 参数时, 不论使用关键字 `class` 或 `typename`, 意义完全相同。

然而 C++ 并不总是把 `class` 和 `typename` 视为等价。有时候你一定得使用 `typename`。为了解其时机, 我们必须先谈谈你可以在 `template` 内指涉 (*refer to*) 的两种名称。

假设我们有个 `template function`，接受一个 STL 兼容容器为参数，容器内持有的对象可被赋值为 `ints`。进一步假设这个函数仅仅只是打印其第二元素值。这是一个无聊的函数，以无聊的方式实现，而且如稍后所言，它甚至不能通过编译。但请暂时漠视那些事，下面是实践这个愚蠢想法的一种方式：

```
template<typename C>
void print2nd(const C& container)           //打印容器内的第二元素
{                                           //注意这不是有效的 C++代码
    if (container.size() >= 2) {
        C::const_iterator iter(container.begin()); //取得第一元素的迭代器
        ++iter;                                   //将 iter 移往第二元素
        int value = *iter;                        //将该元素复制到某个 int
        std::cout << value;                      //打印那个 int
    }
}
```

我在代码中特别强调两个 `local` 变量 `iter` 和 `value`。`iter` 的类型是 `C::const_iterator`，实际是什么必须取决于 `template` 参数 `C`。`template` 内出现的名称如果相依赖于某个 `template` 参数，称之为从属名称 (*dependent names*)。如果从属名称在 `class` 内呈嵌套状，我们称它为嵌套从属名称 (*nested dependent name*)。`C::const_iterator` 就是这样一个名称。实际上它还是个嵌套从属类型名称 (*nested dependent type name*)，也就是个嵌套从属名称并且指涉某类型。

`print2nd` 内的另一个 `local` 变量 `value`，其类型是 `int`。`int` 是一个并不倚赖任何 `template` 参数的名称。这样的名称是谓非从属名称 (*non-dependent names*)。我不知道为什么不叫做独立名称 (*independent names*)。如果你和我一样认为术语 "non-dependent" 令人憎恶，你我之间起了共鸣。但毕竟 "non-dependent" 已被定为这一类名称的术语，所以请和我一样，眨眨眼睛然后顺从它吧。

嵌套从属名称有可能导致解析 (*parsing*) 困难。举个例子，假设我们令 `print2nd` 更愚蠢些，这样起头：

```
template<typename C>
void print2nd(const C& container)
{
    C::const_iterator* x;
    ...
}
```

看起来好像我们声明 `x` 为一个 `local` 变量，它是个指针，指向一个 `C::const_iterator`。但它之所以被那么认为，只因为我们“已经知道”`C::const_iterator` 是个类型。如果 `C::const_iterator` 不是个类型呢？如果 `C` 有个 `static` 成员变量而碰巧被命名为 `const_iterator`，或如果 `x` 碰巧是个 `global`

变量名称呢？那样的话上述代码就不再是声明一个 local 变量，而是一个相乘动作：C::const_iterator 乘以 x。当然啦，这听起来有点疯狂，但却是可能的，而撰写 C++ 解析器的人必须操心所有可能的输入，甚至是这么疯狂的输入。

在我们知道 C 是什么之前，没有任何办法可以知道 C::const_iterator 是否为一个类型。而当编译器开始解析 template print2nd 时，尚未确知 C 是什么东西。C++ 有个规则可以解析 (resolve) 此一歧义状态：如果解析器在 template 中遭遇一个嵌套从属名称，它便假设这名称不是个类型，除非你告诉它是。所以缺省情况下嵌套从属名称不是类型。此规则有个例外，稍后我会提到。

把这些记在心上，现在再次看看 print2nd 起始处：

```
template<typename C>
void print2nd(const C& container)
{
    if (container.size() >= 2) {
        C::const_iterator iter(container.begin());    //这个名称被
        ...                                           //假设为非类型
```

现在应该很清楚为什么这不是有效的 C++ 代码了吧。iter 声明式只有在 C::const_iterator 是个类型时才合理，但我们并没有告诉 C++ 说它是，于是 C++ 假设它不是。若要矫正这个形势，我们必须告诉 C++ 说 C::const_iterator 是个类型。只要紧临它之前放置关键字 typename 即可：

```
template<typename C>                //这是合法的 C++ 代码
void print2nd(const C& container)
{
    if (container.size() >= 2) {
        typename C::const_iterator iter(container.begin());
        ...
    }
}
```

一般性规则很简单：任何时候当你想要在 template 中指涉一个嵌套从属类型名称，就必须在紧临它的前一个位置放上关键字 typename。（再提醒一次，很快我会谈到一个例外。）

typename 只被用来验明嵌套从属类型名称；其他名称不该有它存在。例如下面这个 function template，接受一个容器和一个“指向该容器”的迭代器：

```
template<typename C>                //允许使用 "typename" (或"class")
void f(const C& container,           //不允许使用 "typename"
        typename C::iterator iter); //一定要使用 "typename"
```

上述的 `c` 并不是嵌套从属类型名称（它并非嵌套于任何“取决于 `template` 参数”的东西内），所以声明 `container` 时并不需要以 `typename` 为前导，但 `C::iterator` 是个嵌套从属类型名称，所以必须以 `typename` 为前导。

“`typename` 必须作为嵌套从属类型名称的前缀词”这一规则的例外是，`typename` 不可以出现在 `base classes list` 内的嵌套从属类型名称之前，也不可在 `member initialization list`（成员初值列）中作为 `base class` 修饰符。例如：

```
template<typename T>
class Derived: public Base<T>::Nested {    //base class list 中
public:                                    //不允许 "typename".
    explicit Derived(int x)
    : Base<T>::Nested(x)                  //mem. init. list 中
    {                                     //不允许 "typename".
        typename Base<T>::Nested temp;    //嵌套从属类型名称,
        ...                               //既不在 base class list 中也不在 mem. init. list 中,
    }                                     //作为一个 base class 修饰符需加上 typename.
    ...
};
```

这样的不一致性真令人恼恨，但一旦你有了一些经验，勉勉强强还能接受它。

让我们看看最后一个 `typename` 例子，那是你将在真实程序中看到的代表性例子。假设我们正在撰写一个 `function template`，它接受一个迭代器，而我们打算为该迭代器指涉的对象做一份 `local` 复件（副本）`temp`。我们可以这么写：

```
template<typename IterT>
void workWithIterator(IterT iter)
{
    typename std::iterator_traits<IterT>::value_type temp(*iter);
    ...
}
```

别让 `std::iterator_traits<IterT>::value_type` 吓住了你，那只不过是标准 `traits class`（见条款 47）的一种运用，相当于说“类型为 `IterT` 之对象所指之物的类型”。这个语句声明一个 `local` 变量（`temp`），使用 `IterT` 对象所指物的相同类型，并将 `temp` 初始化为 `iter` 所指物。如果 `IterT` 是 `vector<int>::iterator`，`temp` 的类型就是 `int`。如果 `IterT` 是 `list<string>::iterator`，`temp` 的类型就是 `string`。由于 `std::iterator_traits<IterT>::value_type` 是个嵌套从属类型名称（`value_type` 被嵌套于 `iterator_traits<IterT>` 之内而 `IterT` 是个 `template` 参数），所以我们在它之前放置 `typename`。

如果你认为 `std::iterator_traits<IterT>::value_type` 读起来不畅快, 想象一下打那么长的字又是什么光景。如果你像大多数程序员一样, 认为多打几次这些字实在很恐怖, 那么你应该会想建立一个 `typedef`。对于 `traits` 成员名称如 `value_type` (再次请看条款 47 提供的 `traits` 信息), 普遍的习惯是设定 `typedef` 名称用以代表某个 `traits` 成员名称, 于是常常可以看到类似这样的 `local typedef`:

```
template<typename IterT>
void workWithIterator(IterT iter)
{
    typedef typename std::iterator_traits<IterT>::value_type value_type;
    value_type temp(*iter);
    ...
}
```

许多程序员最初认为把 "typedef typename" 并列颇不合谐, 但它实在是指涉“嵌套从属类型名称”的一个合理附带结果。你很快会习惯它, 毕竟你有强烈的动机——你希望多打几次 `typename std::iterator_traits<IterT>::value_type` 吗?

作为结语, 我应该提到, `typename` 相关规则在不同的编译器上有不同的实践。某些编译器接受的代码原本该有 `typename` 却遗漏了; 原本不该有 `typename` 却出现了; 还有少数编译器 (通常是较旧版本) 根本就拒绝 `typename`。这意味 `typename` 和“嵌套从属类型名称”之间的互动, 也许会在移植性方面带给你某种温和的头疼。

请记住

- 声明 `template` 参数时, 前缀关键字 `class` 和 `typename` 可互换。
- 请使用关键字 `typename` 标识嵌套从属类型名称; 但不得在 `base class lists` (基类列) 或 `member initialization list` (成员初值列) 内以它作为 `base class` 修饰符。

条款 43: 学习处理模板化基类内的名称

Know how to access names in templated base classes.

假设我们需要撰写一个程序, 它能够传送信息到若干不同的公司去。信息要不译成密码, 要不就是未经加工的文字。如果编译期间我们有足够信息来决定哪一个信息传至哪一家公司, 就可以采用基于 `template` 的解法: